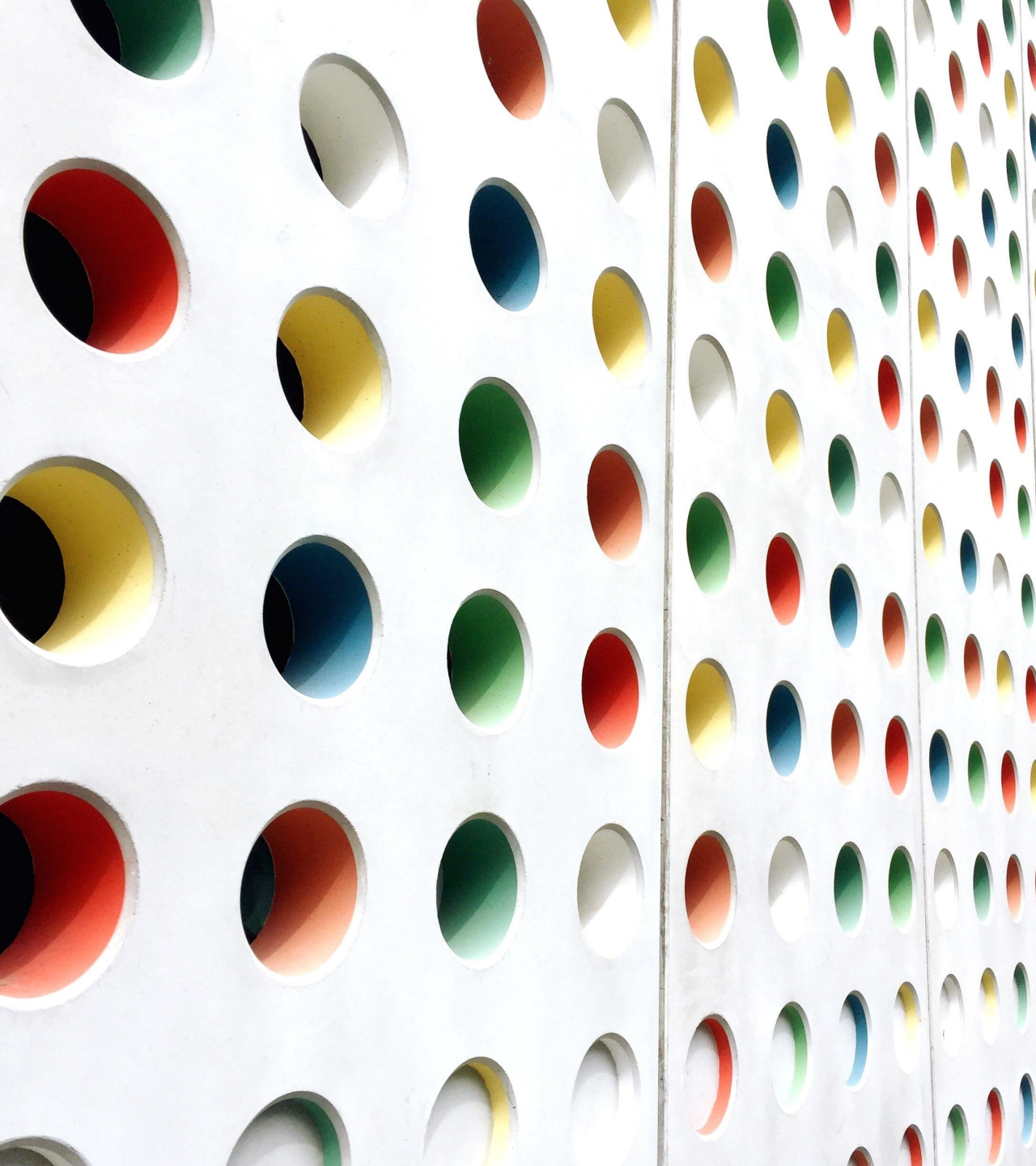




SPARQL

BY

SANA RIZWAN

Conventions

Red text means:

“This is a core part of the SPARQL syntax or language.”

Blue text means:

“This is an example of query-specific text or values that might go into a SPARQL query.”

- SPARQL is a query language/engine for RDF graphs.
- An RDF graph is a set of triples. A flexible and extensible way to represent information about WWW resources
- A concept similar to SQL for data bases. A W3C standard query language to fetch data from distributed Semantic Web data models.
- Can query a triple store or data on the Web (at a given URL). It provides facilities to:
 - extract information in the form of URIs, blank nodes, plain and typed literals.
 - extract RDF subgraphs.
 - construct new RDF graphs based on information in the queried graphs

Spqrql – FIRST EXAMPLE

```
prefix sch-ont: <http://education.data.gov.uk/def/school/>
SELECT ?name WHERE { ?school a sch-ont:School;
    sch-ont:establishmentName ?name;
    sch-ont:districtAdministrative <http://statistics.data.gov.uk/id/local-authority-district/00AA> ;
}
ORDER BY ?name
```

- If executed on the UK government's **Open Semantic Database**, will return the names of all the schools in the UK in administrative district 00AA, and order the results in alphabetical order (<http://data.gov.uk/sparql>)
- **SPARQL Is Similar To SQL**
 - Like SQL, SPARQL selects data from the query data set by using a SELECT statement to determine which subset of the selected data is returned. Also, SPARQL uses a WHERE clause to define graph patterns to find a match for in the query data set.

General clause

PREFIX (Namespace Prefixes)
e.g. PREFIX plant: <http://www.linkeddatatools.com/plants>
SELECT (Result Set)
e.g. SELECT ?name
FROM (Data Set)
e.g. FROM <http://www.linkeddatatools.com/plantsdata/plants.rdf>
WHERE (Query Triple Pattern)
e.g. WHERE { ?planttype plant:planttype ?name }
ORDER BY, DISTINCT etc (Modifiers)
e.g. ORDER BY ?name

Data set – triple [subject, predicate, object]

Subject	Predicate	Object
http://www.linkeddatatools.com/plants#magnolia	http://www.linkeddatatools.com/plants#family	Magnoliaceae
http://www.linkeddatatools.com/plants#african_lilly	http://www.linkeddatatools.com/plants#family	Liliaceae
http://www.linkeddatatools.com/plants#silvertop	http://www.linkeddatatools.com/plants#family	Aralianae
http://www.linkeddatatools.com/plants#velvetleaf	http://www.linkeddatatools.com/plants#family	Malvaceae
http://www.linkeddatatools.com/plants#manglietia	http://www.linkeddatatools.com/plants#family	Magnoliaceae

Example: triple data containing a variety of shrubs and plants, and their family names

Select All Data

PREFIX plants: <<http://www.linkeddatatools.com/plants>>

SELECT * WHERE

{ ?name plants:family ?family }

- select all the plant URIs (subjects) and plant family names (literal-type objects) from the data above;
- the wildcard '*' in SPARQL, similar to SQL, will return all the mapped data in the result set;
- as we have stated two variables ?name and ?family, the query will return both these declared query variables in our result set, according to the subjects, predicates or objects they're mapped to;

Nuts & Bolts

URIs

Write full URIs:

```
<http://this.is.a/full/URI/written#out>
```

Abbreviate URIs with prefixes:

```
PREFIX foo: <http://this.is.a/URI/prefix#>
```

```
... foo:bar ...
```

```
    ⇨ http://this.is.a/URI/prefix#bar
```

Shortcuts:

```
    a  ⇨ rdf:type
```

Literals

Plain literals:

```
"a plain literal"
```

Plain literal with language tag:

```
"bonjour"@fr
```

Typed literal:

```
"13"^^xsd:integer
```

Shortcuts:

```
true ⇨ "true"^^xsd:boolean
```

```
3    ⇨ "3"^^xsd:integer
```

```
4.2  ⇨ "4.2"^^xsd:decimal
```

Variables

Variables:

```
?var1, ?anotherVar, ?and_one_more
```

Comments

Comments:

```
# Comments start with a '#'
```

```
# continue to the end of the line
```

Triple Patterns

Match an exact RDF triple:

```
ex:myWidget ex:partNumber "XY24Z1" .
```

Match one variable:

```
?person foaf:name "Lee Feigenbaum" .
```

Match multiple variables:

```
conf:SemTech2009 ?property ?value .
```


Common Prefixes

prefix...

...stands for

rdf:

<http://xmlns.com/foaf/0.1/>

rdfs:

<http://www.w3.org/2000/01/rdf-schema#>

owl:

<http://www.w3.org/2002/07/owl#>

xsd:

<http://www.w3.org/2001/XMLSchema#>

dc:

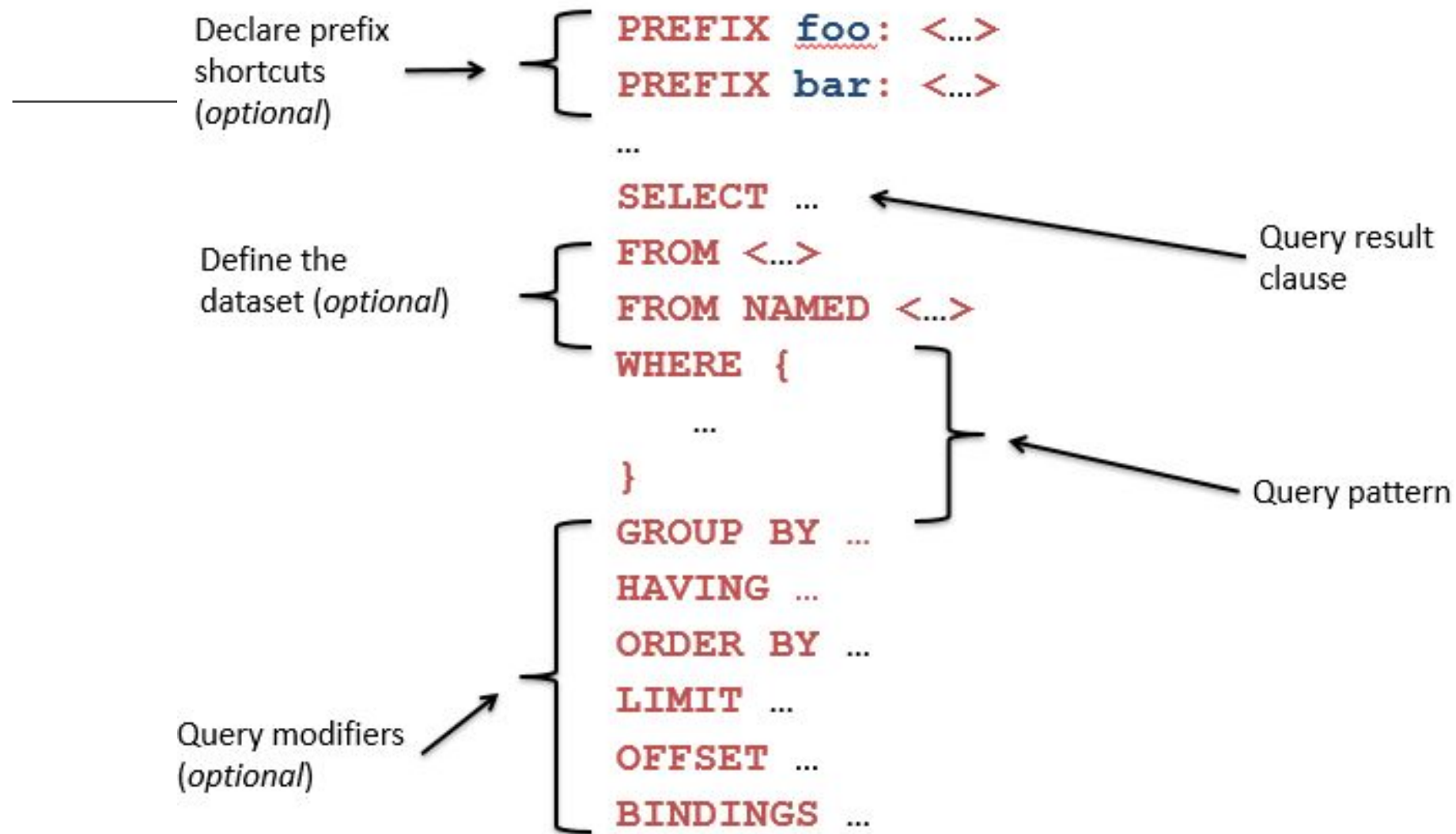
<http://purl.org/dc/elements/1.1/>

foaf:

<http://xmlns.com/foaf/0.1/>

More common prefixes at <http://prefix.cc>

Anatomy of a Query



4 Types of SPARQL Queries

SELECT queries

Project out specific variables and expressions:

```
SELECT ?c ?cap (1000 * ?people AS ?pop)
```

Project out all variables:

```
SELECT *
```

Project out distinct combinations only:

```
SELECT DISTINCT ?country
```

Results in a table of values (in [XML](#) or [JSON](#)):

?c	?cap	?pop
ex:France	ex:Paris	63,500,000
ex:Canada	ex:Ottawa	32,900,000
ex:Italy	ex:Rome	58,900,000

CONSTRUCT queries

Construct RDF triples/graphs:

```
CONSTRUCT {  
  ?country a ex:HolidayDestination ;  
  ex:arrive_at ?capital ;  
  ex:population ?population .  
}
```

Results in RDF triples (in any RDF serialization):

```
ex:France a ex:HolidayDestination ;  
  ex:arrive_at ex:Paris ;  
  ex:population 635000000 .  
ex:Canada a ex:HolidayDestination ;  
  ex:arrive_at ex:Ottawa ;  
  ex:population 329000000 .
```

ASK queries

Ask whether or not there are any matches:

```
ASK
```

Result is either “true” or “false” (in [XML](#) or [JSON](#)):

```
true, false
```

DESCRIBE queries

Describe the resources matched by the given variables:

```
DESCRIBE ?country
```

Result is RDF triples (in any RDF serialization) :

```
ex:France a geo:Country ;  
  ex:continent geo:Europe ;  
  ex:flag <http://.../flag-france.png> ;  
  ...
```

Combining SPARQL Graph Patterns

Consider **A** and **B** as graph patterns.

A Basic Graph Pattern – one or more triple patterns

A . B

⇒ Conjunction. Join together the results of solving A and B by matching the values of any variables in common.

Optional Graph Patterns

A OPTIONAL { B }

⇒ Left join. Join together the results of solving A and B by matching the values of any variables in common, if possible. Keep all solutions from A whether or not there's a matching solution in B

Combining SPARQL Graph Patterns

Consider **A** and **B** as graph patterns.

Either-or Graph Patterns

{ A } UNION { B }

⇒ Disjunction. Include both the results of solving A and the results of solving B.

“Subtracted” Graph Patterns (SPARQL 1.1)

A MINUS { B }

⇒ Negation. Solve A. Solve B. Include only those results from solving A that are *not compatible* with any of the results from B.

SPARQL Subqueries [SPARQL 1.1]

Consider **A** and **B** as graph patterns.

```
A .  
{  
  SELECT ...  
  WHERE {  
    B  
  }  
}  
C .
```

⇒ Join the results of the subquery with the results of solving A and C.

SPARQL Filters

SPARQL **FILTERS** eliminate solutions that do not cause an expression to evaluate to true.

A . B . FILTER (...expr...)

Place **FILTERS** in a query inline within a basic graph pattern

Category	Functions / Operators	Examples
Logical	!, &&, , =, !=, <, <=, >, >=	?hasPermit ?age < 25
Math	+, -, *, /	?decimal * 10 > ?minPercent
Existence (SPARQL 1.1)	EXISTS, NOT EXISTS	NOT EXISTS { ?p foaf:mbox ?email }
SPARQL tests	isURI, isBlank, isLiteral, bound	isURI(?person) !bound(?person)
Accessors	str, lang, datatype	lang(?title) = "en"
Miscellaneous	sameTerm, regex	regex(?ssn, "\\d{3}-\\d{2}-\\d{4}")

Aggregates [SPARQL 1.1]

1. Partition results into groups based on the expression(s) in the **GROUP BY** clause
2. Evaluate projections and aggregate functions in **SELECT** clause to get one result per group
3. Filter aggregated results via the **HAVING** clause

	?key	?val	?other1
[	1	4	...
	1	4	...
	2	5	...
	2	4	...
]	2	10	...
	2	2	...
	2	1	...
	3	3	...

↓

?key	?sum_of_val
1	8
2	22
3	3

↓

?key	?sum_of_val
1	8
3	3

SPARQL 1.1 includes: **COUNT**, **SUM**, **AVG**, **MIN**, **MAX**, **SAMPLE**, **GROUP_CONCAT**

Property Paths [SPARQL 1.1]

Property paths allow triple patterns to match arbitrary-length paths through a graph

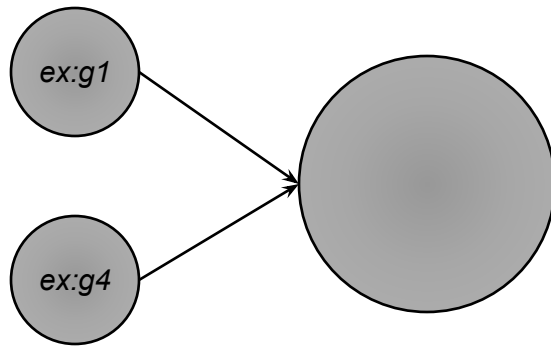
Predicates are combined with regular-expression-like operators:

Construct	Meaning
<code>path1/path2</code>	Forwards path (<code>path1</code> followed by <code>path2</code>)
<code>^path1</code>	Backwards path (object to subject)
<code>path1 path2</code>	Either <code>path1</code> or <code>path2</code>
<code>path1*</code>	<code>path1</code> , repeated zero or more times
<code>path1+</code>	<code>path1</code> , repeated one or more times
<code>path1?</code>	<code>path1</code> , optionally
<code>path1{m,n}</code>	At least <code>m</code> and no more than <code>n</code> occurrences of <code>path1</code>
<code>path1{n}</code>	Exactly <code>n</code> occurrences of <code>path1</code>
<code>path1{m,}</code>	At least <code>m</code> occurrences of <code>path1</code>
<code>path1{,n}</code>	At most <code>n</code> occurrences of <code>path1</code>

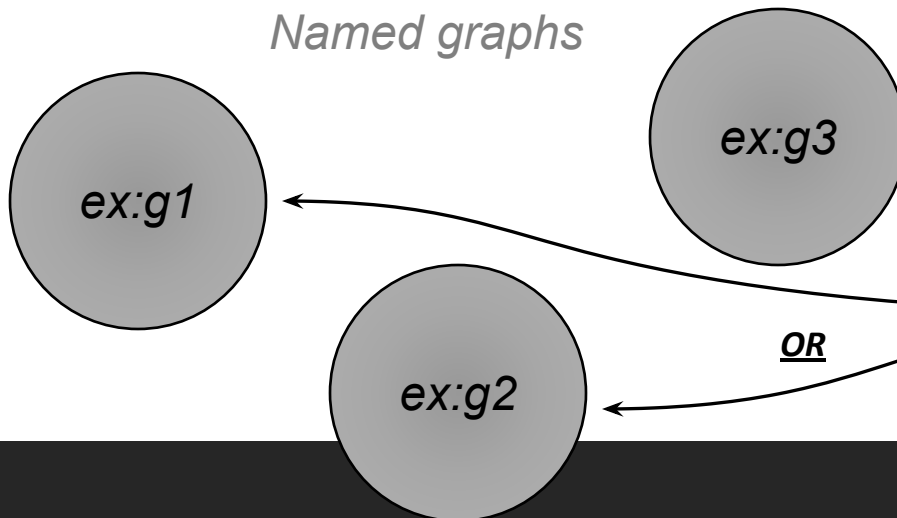
RDF Datasets

A SPARQL query a *default graph* (normally) and zero or more *named graphs* (when inside a **GRAPH** clause).

Default graph
(the merge of zero or more graphs)



Named graphs



```
PREFIX ex: <...>
SELECT ...
FROM ex:g1
FROM ex:g4
FROM NAMED ex:g1
FROM NAMED ex:g2
FROM NAMED ex:g3
WHERE {
  ... A ...
  GRAPH ex:g3 {
    ... B ...
  }
  GRAPH ?g {
    ... C ...
  }
}
```

OR

OR

SPARQL in 11 mins

<https://www.youtube.com/watch?v=FvGndkpa4K0>

SPARQL Resources

- The SPARQL Specification
 - <http://www.w3.org/TR/rdf-sparql-query/>
- SPARQL implementations
 - <http://esw.w3.org/topic/SparqlImplementations>
- SPARQL endpoints
 - <http://esw.w3.org/topic/SparqlEndpoints>
- SPARQL Frequently Asked Questions
 - <http://www.thefigtrees.net/lee/sw/sparql-faq>
- SPARQL Working Group
 - <http://www.w3.org/2009/sparql/wiki/>
- Common SPARQL extensions
 - <http://esw.w3.org/topic/SPARQL/Extensions>